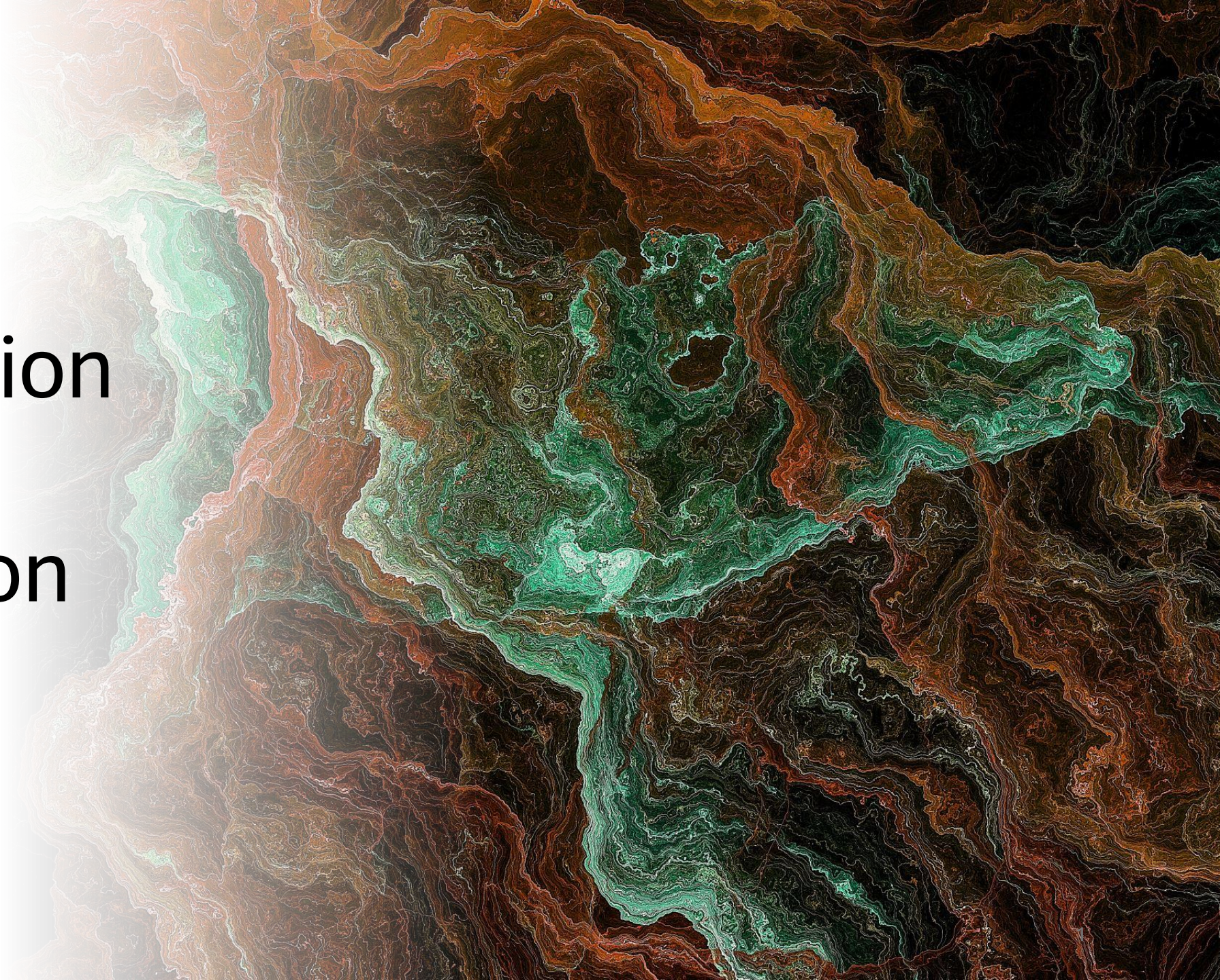
Inventory Optimization and Automation

By Shantanu Patil

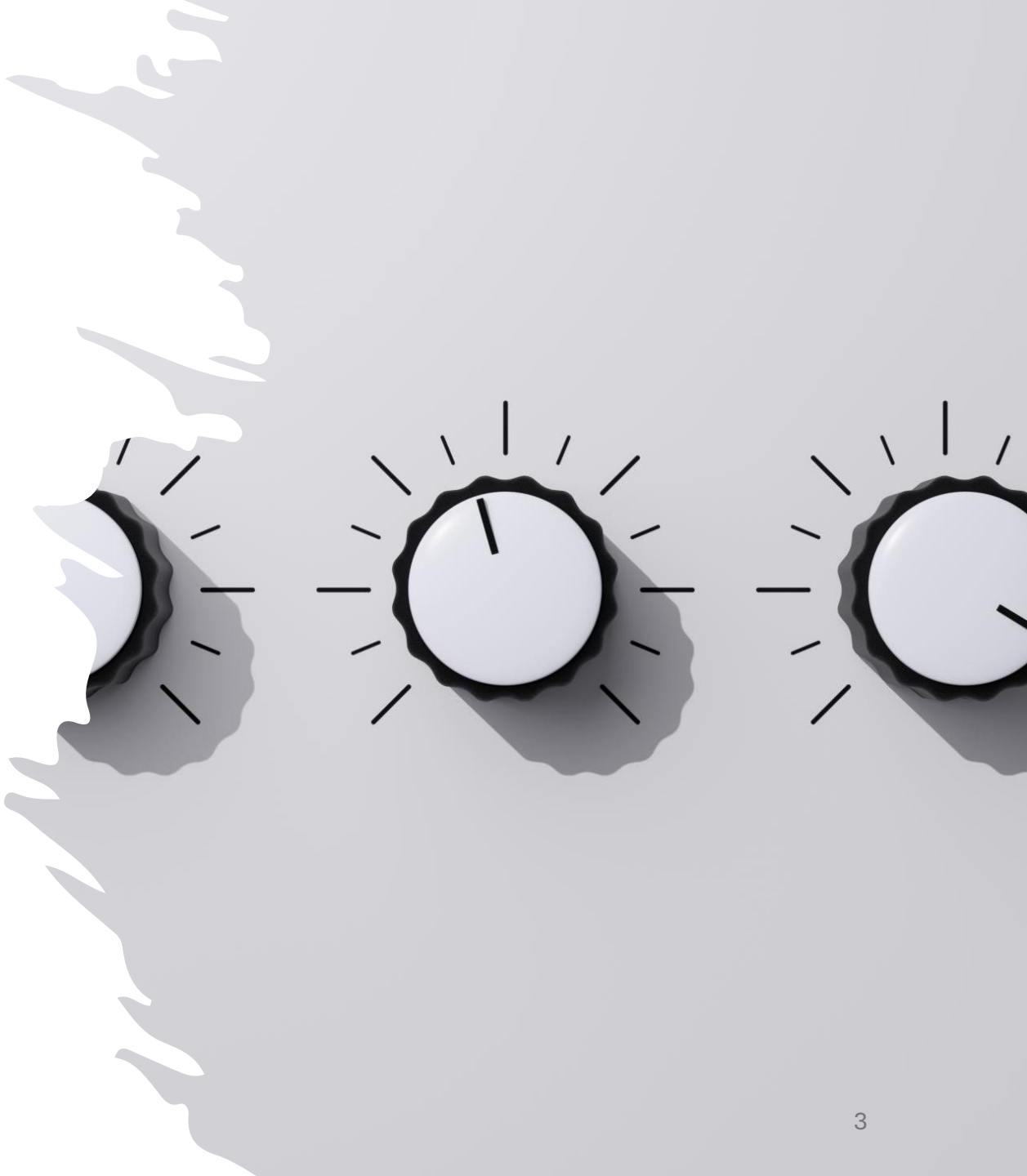


Contents

- Inventory Management
- Simulation Model
- Prototype of Autonomous Robot
- Shipping Optimization
- Conclusion

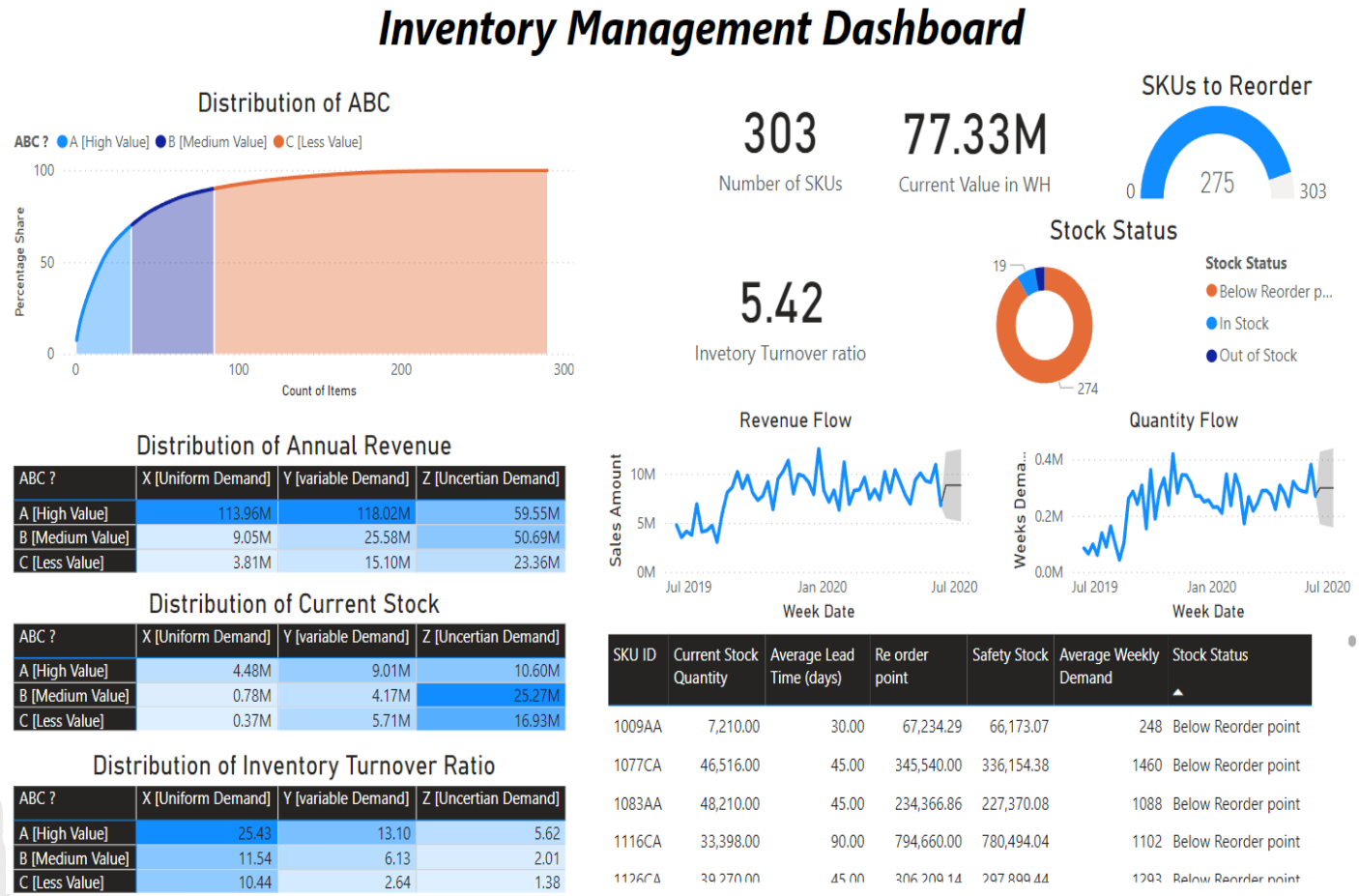
Inventory Management Dashboard - Power BI

A comprehensive dashboard to manage and analyze inventory levels and understand the SKUs. (Sample Data)



Interactive Dashboard Insights:

- **Inventory Overview:**
  - **Number of SKUs:** 303
  - **Current Warehouse Value:** $77.33M
  - **Inventory Turnover Ratio:** 5.42 (indicating efficient stock movement)
- **Classification Insights:**
  - **ABC Classification:** Focus on high-value items (A) for maximizing revenue, while optimizing stock levels of medium (B) and low-value (C) items.
  - **XYZ Classification:** Manage uniform (X), variable (Y), and uncertain (Z) demand effectively to minimize stockouts and overstock.
- **Stock Status:**
  - 275 SKUs need reordering, highlighting critical restocking needs.
- **Trends:**
  - **Revenue Flow:** Consistent sales with occasional peaks, useful for forecasting.
  - **Quantity Flow:** Steady demand, aiding in inventory planning.



Distribution of Annual Revenue

ABC ?	X [Uniform Demand]	Y [variable Demand]	Z [Uncertian Demand]
A [High Value]	113.96M	118.02M	59.55M
B [Medium Value]	9.05M	25.58M	50.69M
C [Less Value]	3.81M	15.10M	23.36M

Distribution of Current Stock

ABC ?	X [Uniform Demand]	Y [variable Demand]	Z [Uncertian Demand]
A [High Value]	4.48M	9.01M	10.60M
B [Medium Value]	0.78M	4.17M	25.27M
C [Less Value]	0.37M	5.71M	16.93M

Distribution of Inventory Turnover Ratio

ABC ?	X [Uniform Demand]	Y [variable Demand]	Z [Uncertian Demand]
A [High Value]	25.43	13.10	5.62
B [Medium Value]	11.54	6.13	2.01
C [Less Value]	10.44	2.64	1.38

SKU ID	Current Stock Quantity	Average Lead Time (days)	Re order point	Safety Stock	Average Weekly Demand	Stock Status
1009AA	7,210.00	30.00	67,234.29	66,173.07	248	Below Reorder point
1077CA	46,516.00	45.00	345,540.00	336,154.38	1460	Below Reorder point
1083AA	48,210.00	45.00	234,366.86	227,370.08	1088	Below Reorder point
1116CA	33,398.00	90.00	794,660.00	780,494.04	1102	Below Reorder point
1126CA	39,270.00	45.00	306,209.14	297,899.44	1293	Below Reorder point

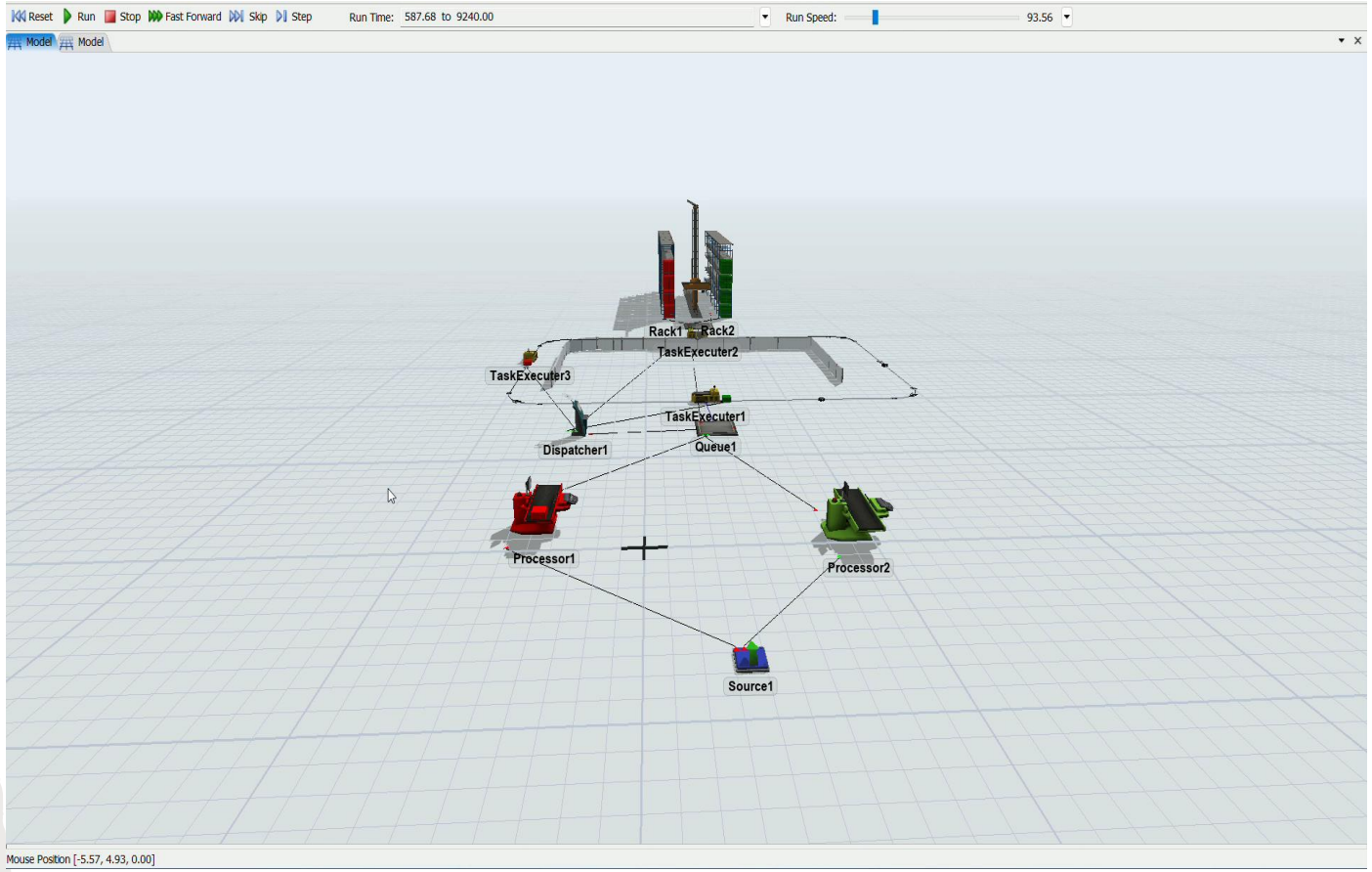
Simulation Model - Flexsim

Simulating the process of collection, transportation (AGVs), and racking using ASRS vehicles.

Simple Simulation Model Video:

- **Warehouse Process Simulation:**
  - Simulate the flow of goods from source, distribution, and racking.
  - Optimize the placement of warehouse racks, ASRS systems, and AGVs (Automated Guided Vehicles).

- **Efficiency Gains:**
  - Identify and mitigate bottlenecks.
  - Improve throughput and reduce handling time.
  - Safe, minimum human interaction, low margin of error.
  - E80 AGVs can move autonomously, streamline, and improve the intralogistics operations

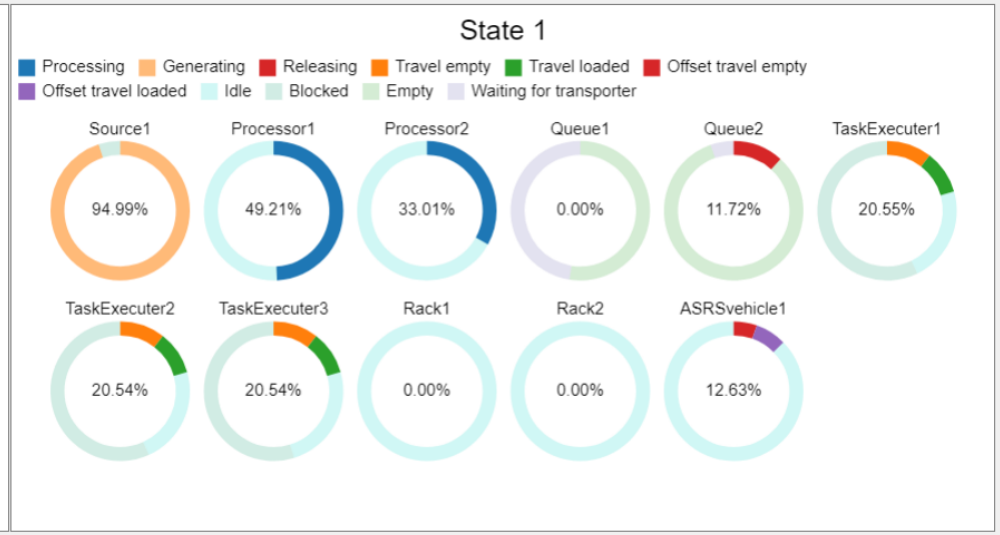


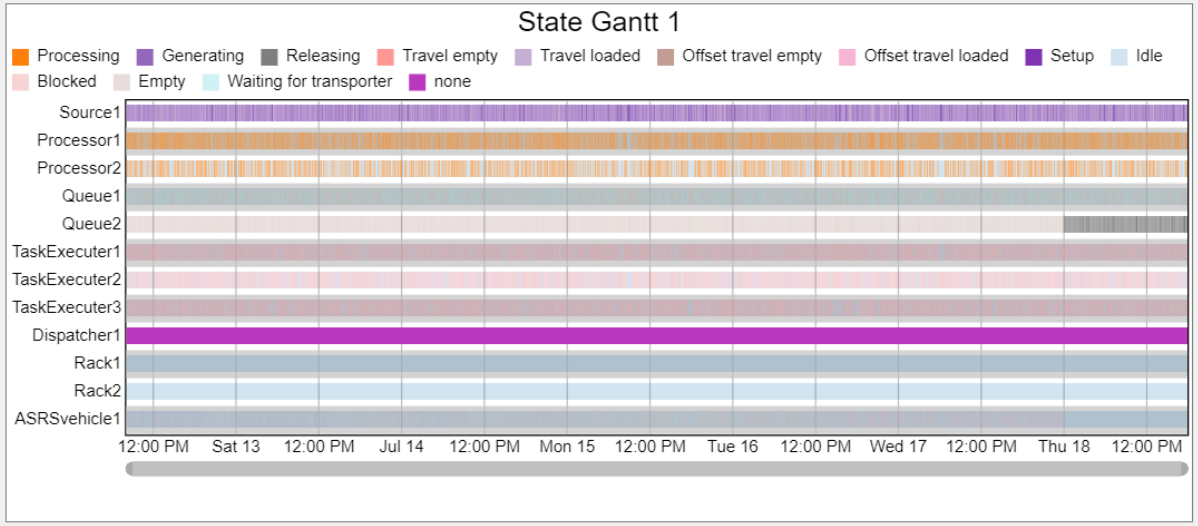
Flexsim Simulation Model:

- **Warehouse Process Simulation:**
  - **Average Work in Progress (WIP):** Insights into the average WIP levels show the efficiency of various objects in the warehouse. For instance:
    - **Processor1 and Processor2:** WIP levels at 0.49 and 0.33 respectively, indicating moderate usage.
    - **Queue2:** High WIP level of 25.32, suggesting a potential bottleneck that needs addressing.
    - **Rack1 and Rack2:** High average content of 863.39 and 865.69 respectively, which might indicate high storage utilization.
  - **State Analysis:**
    - Visual representation of object states (Processing, Idle, Blocked) helps identify inefficiencies and optimize process flow.
    - **Gantt Chart:** Detailed view of the operation timeline, highlighting periods of activity and idle times for each object.

Average Content

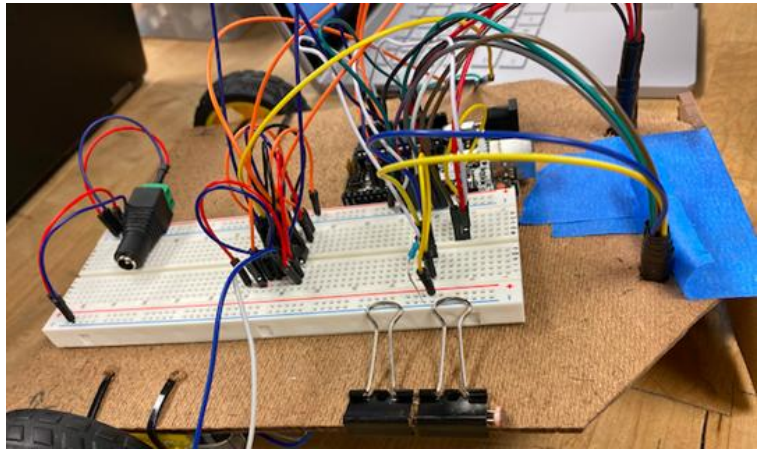
Object	AverageWIP
Source1	0.00
Processor1	0.49
Processor2	0.33
Queue1	0.55
Queue2	25.32
TaskExecuter1	0.60
TaskExecuter2	0.60
TaskExecuter3	0.58
Dispatcher1	0.00
Rack1	863.39
Rack2	865.69
ASRSvehicle1	0.07

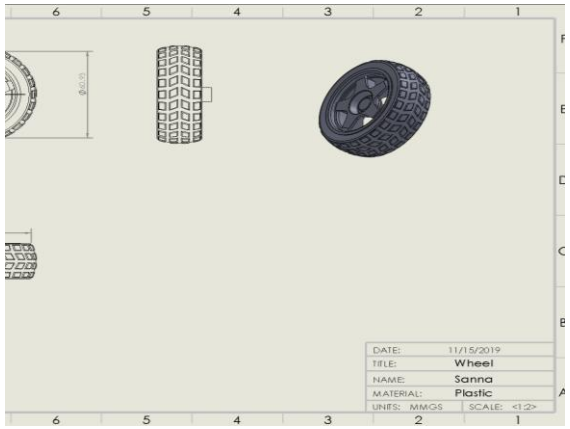


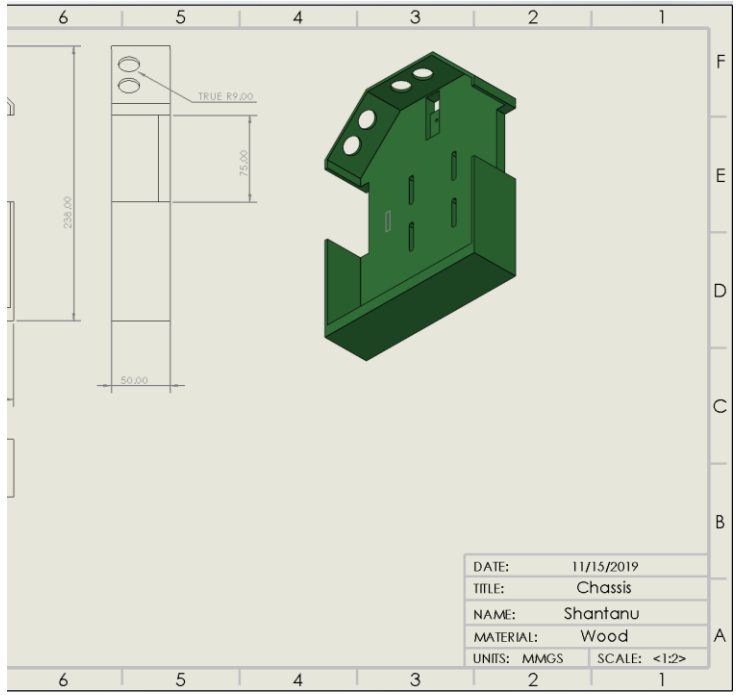


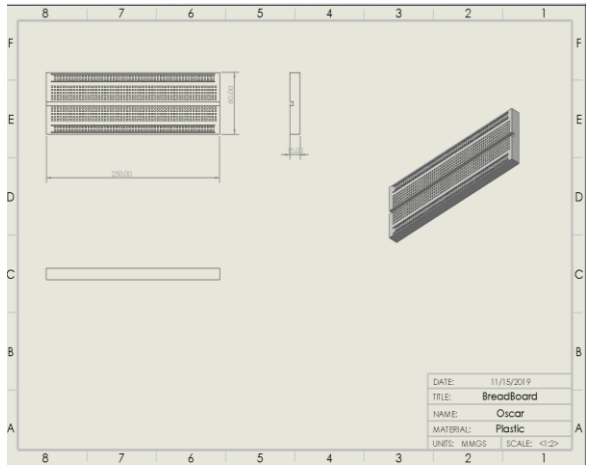
Prototype & 3D Drawing of Autonomous Robot - SolidWorks

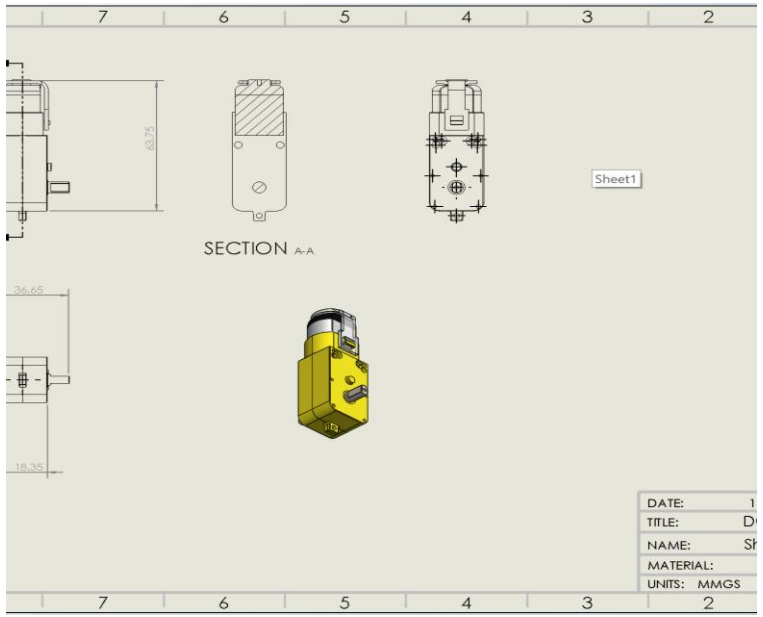
A detailed 3D model of the autonomous robot designed and built.

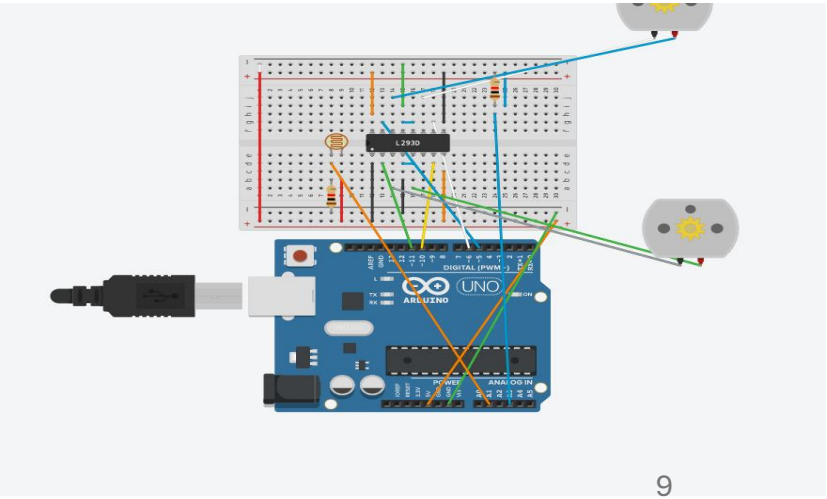



DATE: 11/15/2019
TITLE: Wheel
NAME: Sanna
MATERIAL: Plastic
UNITS: MMGS
SCALE: <1:2>


TRUE R9.00
75.00
238.00
50.00
DATE: 11/15/2019
TITLE: Chassis
NAME: Shantanu
MATERIAL: Wood
UNITS: MMGS
SCALE: <1:2>


DATE: 11/15/2019
TITLE: BreadBoard
NAME: Oscar
MATERIAL: Plastic
UNITS: MMGS
SCALE: <1:2>


63.75
SECTION A-A
36.65
18.35
Sheet1
DATE:
TITLE:
NAME:
MATERIAL:
UNITS: MMGS

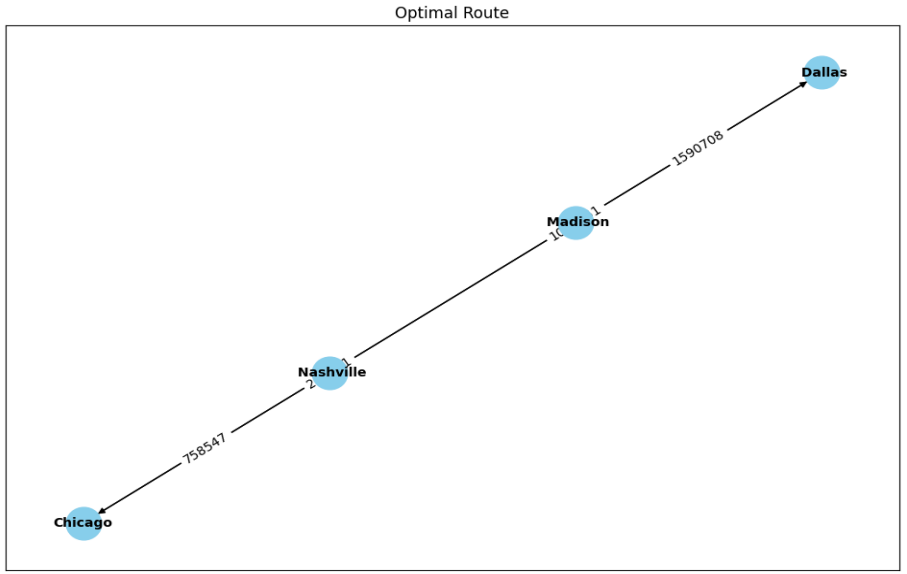

L2930
ARDUINO
UNO
DIGITAL (PWM~)
POWER
ANALOG IN

Optimizing Shipping Routes - Python

Creating a python script to find the best transportation route.

Python Script for Optimal Routing:

- **Google Maps API Integration:**
  - Script utilizes real-time data to find the shortest and fastest routes for transporting materials.
  - Considers traffic conditions, distances, and time savings.
- **Optimization with Pyomo:**
  - Uses linear programming to solve the traveling distance problem, ensuring the most efficient route.
  - Ensures each location is visited once and returns to the starting point, minimizing travel distance and time.
  - Reducing shipping cost.
- **Plotting Optimal Route:**
  - The script also generates a visual representation of the optimal route, aiding in better planning and execution.




```python
    print(distance_matrix)
    print(number_of_places)

    # Optimization with Pyomo
    n = number_of_places  # number of cities
    model = ConcreteModel()
    model.M = RangeSet(1, n)
    model.N = RangeSet(1, n)
    model.U = RangeSet(2, n)
    model.x = Var(model.N, model.M, within=Binary)
    model.u = Var(model.N, within=NonNegativeIntegers, bounds=(0, n-1))

    def distance_matrix_init(model, i, j):
        return distance_matrix[i-1][j-1]

    model.c = Param(model.N, model.M, initialize=distance_matrix_init)

    def obj_func(model):
        return sum(model.x[i,j] * model.c[i,j] for i in model.N for j in model.M)

    model.objective = Objective(rule=obj_func, sense=minimize)

    def rule_const1(model, M):
        return sum(model.x[i, M] for i in model.N if i != M) == 1

    model.const1 = Constraint(model.M, rule=rule_const1)

    def rule_const2(model, N):
        return sum(model.x[N, j] for j in model.M if j != N) == 1

    model.rest2 = Constraint(model.N, rule=rule_const2)

    def rule_const3(model, i, j):
        if i != j:
            return model.u[i] - model.u[j] + model.x[i,j] * n <= n-1
        else:
            return model.u[i] - model.u[i] == 0

    model.rest3 = Constraint(model.U, model.N, rule=rule_const3)

    solver = SolverFactory('glpk', executable='/usr/bin/glpsol')
    result = solver.solve(model, tee=False)

    print(result)

    for i in model.x.keys():
        if model.x[i]() != 0 and model.x[i]() is not None:
            print(i, '--', model.x[i]())

    # Plotting the optimal route
    plot_route(addresses, distance_matrix, model)

    return [distance_matrix, number_of_places]

if __name__ == '__main__':
    distance_matrix, number_of_places = main()
```

Integrated Approach

Combining data-driven insights from Power BI, process optimization from Flexsim, automation through AGVs, and efficient routing with Python scripting provides a holistic solution.

These tools collectively enhance the E80 group's client's inventory management, streamline warehouse operations, and optimize logistics. And by understanding their needs we can provide with the right solution and transform their Warehouse Management to a more efficient and autonomous system.

E80 Group Inc.

With E80 Group Inc. comprehensive range of products we can enhance the efficiency, accuracy, and safety of warehouse and logistics operations.

By leveraging advanced automation technologies, we can help businesses streamline their material handling processes, reduce labor costs, improve overall productivity, and most importantly increase safety.

Promoting innovation for sustainable growth and creating tailor-made intralogistics solutions.

“Everything can be stopped, except the strength of a Dream!”